FINAL-YEAR PROJECT RESEARCH GUIDE

Usage-Aware Memory Retention for Long-Term AI Agents

A guide-ready explanation of the architecture, equations, implementation, originality, limitations, and demonstration workflow.

UAMR Research Project

FastAPI | Ollama | Qwen3 | Mem0 | ChromaDB | SQLite | React

Prepared as a complete technical explanation for project-guide discussion and final-year demonstration.

Contents

1. Project title 7

The title has three important parts: 7

The main research question is: 7

2. Short explanation for your guide 7

You can begin with this: 7

3. Why this project is needed 7

Suppose the agent has: 7

4. System architecture 8

The project contains the following components: 8

SQLite research database 8

Responsibilities 8

Ollama and Qwen3 8

Mem0 8

It handles: 8

ChromaDB 9

ChromaDB stores: 9

SQLite research database 9

SQLite stores lightweight research metadata: 9

UAMR 9

5. How one chat request works 9

When the user submits a message: 9

6. Research instrumentation 10

Why instrumentation was required 10

For example: 10

Retrieval event 10

For every retrieved memory, the system records: 10

For example: 10

Aggregate usage data 10

The exact interception happens in: 11

7. The four UAMR signals 11

UAMR uses four signals: 11

7.1 Retrieval frequency - F(m) 11

The formula is: 11

Where: 11

Name of the mathematical technique . . . 11
Why smoothing is needed . . . 11
Without smoothing: . . . 11
With smoothing: . . . 12
Interpretation . . . 12
Function used . . . 12
The calculation is performed inside: . . . 12

7.2 Recency - R(m) 12

The formula is: . . . 12
Where: . . . 12
Name of the equation . . . 12
It is commonly used in: . . . 12
Why we used exponential decay . . . 12
Exponential decay provides a smooth reduction: . . . 13
Meaning of tau . . . 13
A small value: . . . 13
A larger value: . . . 13
The default in our project is: . . . 13
Never-retrieved memories . . . 13
Function used . . . 13
Implemented in: . . . 13
using: . . . 13

7.3 Retrieval quality - Q(m) 13

For every retrieval, we calculate: . . . 13
Where: . . . 13
The historical score is: . . . 13
If a memory has never been retrieved: . . . 14
Name of the technique . . . 14
Why rank is important . . . 14
At rank 1: . . . 14
At rank 5: . . . 14
Why similarity alone is insufficient . . . 14
Functions used . . . 14
The average is calculated in: . . . 14
using: . . . 14

7.4 Redundancy - D(m) 14

Name of the similarity equation . . . 15
Typical interpretation: . . . 15
Redundancy threshold . . . 15
The default threshold is: . . . 15

If similarity is below the threshold: . . . 15
If similarity is above the threshold: . . . 15
Name of this transformation . . . 15
Example . . . 15
Suppose: . . . 15
Then: . . . 15
Why compare only against higher-value memories? . . . 15
Imagine two almost identical memories: . . . 15
Instead: . . . 16
Function used . . . 16
Implemented in: . . . 16
Cosine similarity is calculated in: . . . 16

8. Final retention equation **16**

The final equation is: . . . 16
Where: . . . 16
Default values: . . . 16
Name of the equation . . . 16
This kind of construction is used in: . . . 16
Why three positive terms and one negative term? . . . 17
Real calculation example . . . 17
Suppose a memory has: . . . 17
Using the defaults: . . . 17
Therefore: . . . 17

9. How we developed our own equation **17**

Our contribution is: . . . 17
The equation-design process . . . 18
Step 1: Define the objective . . . 18
We wanted to estimate: . . . 18
Step 2: Identify observable evidence . . . 18
We selected: . . . 18
Step 3: Keep signals on comparable scales . . . 18
Step 4: Combine positive evidence . . . 18
We used a weighted sum: . . . 18
Step 5: Add a duplication penalty . . . 18
We subtracted: . . . 18
Step 6: Separate scoring from the budget decision . . . 18
This separation is important: . . . 18
Step 7: Add cold-start protection . . . 18
Step 8: Make everything configurable . . . 19

10. Cold-start grace period **19**

A new memory may initially have: . . . 19
Therefore: . . . 19
Default: . . . 19

11. Memory-budget algorithm **19**

After scores are calculated: . . . 19
2. Calculate the target: . . . 19
Function used . . . 19
Implemented in: . . . 19
Example . . . 19
For: . . . 19
The target is: . . . 20
Therefore: . . . 20

12. ACTIVE and PRUNED do not mean deletion **20**

This is important because: . . . 20

13. Deterministic explanations **20**

Examples: . . . 20
This makes the research output: . . . 20
The function is: . . . 20

14. PCA vector memory map **21**

Name of the technique . . . 21
We reduce: . . . 21
Each memory becomes: . . . 21
Mean centering . . . 21
For every dimension, subtract the mean: . . . 21
Singular Value Decomposition . . . 21
The implementation uses: . . . 21
Projection . . . 21
Why PCA? . . . 21
We chose PCA because it is: . . . 21
Important limitation . . . 22
Therefore: . . . 22
The implementation is in: . . . 22

15. Why PCA is cached **22**

Changing the budget affects only policy states: . . . 22
Only the following reach the browser: . . . 22

16. Memory Lab explanation **22**

It shows: . . . 22
The redundancy rate is: . . . 23

Memory table . . . 23
Every row shows: . . . 23
Filters include: . . . 23
Detail drawer . . . 23
Clicking a row or PCA point shows: . . . 23
Frontend implementation: . . . 24

17. Policy Engine explanation **24**

It displays: . . . 24
Then the final formula: . . . 24
Budget simulator . . . 24
The slider supports: . . . 24
Frontend implementation: . . . 25

18. Important backend classes and functions **25**

19. What is original in the project? **25**

A good academic answer is: . . . 25
The project-specific contributions are: . . . 25
Instead say: . . . 26

20. Current limitations **26**

21. Future experimental work **26**

Possible future work includes: . . . 26
Useful evaluation metrics could include: . . . 27

22. Suggested final presentation statement **27**

1. Project title

Usage-Aware Memory Retention for Long-Term AI Agents

The title has three important parts:

- Usage-aware: The system observes how memories are actually used-how often they are retrieved, how recently they were retrieved, their retrieval rank, and semantic similarity.
- Memory retention: The system decides which memories should remain active when memory capacity is limited.
- Long-term AI agent: The agent remembers information across multiple conversations using Mem0 and ChromaDB.

The main research question is:

> **If an AI agent has more memories than it can keep active, how can it retain the memories most likely to be useful in future conversations?**

2. Short explanation for your guide

You can begin with this:

> **Our project is a local AI agent with long-term memory. The agent uses Ollama and Qwen3 for response generation, Mem0 for memory management, and ChromaDB for storing memory embeddings.**

Our research contribution is UAMR-Usage-Aware Memory Retention. Instead of keeping every memory active forever or removing memories only based on age, UAMR evaluates how useful each memory has been.

It calculates four signals: retrieval frequency, recency, retrieval quality, and redundancy. These signals are combined into a final retention score. When a memory budget such as 50% is selected, the system ranks all memories and marks the most useful ones ACTIVE and the remaining ones PRUNED.

This is currently a non-destructive simulation. PRUNED does not mean deleted. It only represents what the policy would remove under that budget.

We also built an instrumentation database that records every retrieval event, and a research interface that visualizes the scores, decisions, retrieval history, and a PCA projection of the memory embeddings.

3. Why this project is needed

A long-running AI agent may continuously create new memories.

Suppose the agent has:

```
N = 10,000 memories
```

Keeping every memory active can create several problems:

- Retrieval becomes slower.
- More irrelevant memories may enter the prompt.
- Similar or duplicate memories consume storage.
- Old, unused information competes with useful information.
- The agent may retrieve noisy context.

- Memory maintenance becomes expensive.

Therefore, we introduce an active-memory budget:

```
B < N
```

For example, with 100 memories and a 50% budget:

```
N = 100
```

```
B = 0.50
```

```
Active-memory target = ceil(100 x 0.50) = 50
```

The system must identify which 50 memories are most valuable.

4. System architecture

The project contains the following components:

User

v

React web interface / CLI

v

FastAPI

v

Existing Agent

|- LLMProvider -> Ollama -> qwen3:4b

`- MemoryEngine -> Mem0 -> ChromaDB

v

UsageTracker

v

SQLite research database

v

UAMR Policy

Responsibilities

Ollama and Qwen3

Qwen3 generates the natural-language response.

The project runs locally, so prompts and memories do not need to be sent to an external cloud model.

Mem0

Mem0 is the baseline memory system.

It handles:

- Memory creation
- Memory retrieval

- Memory updating
- Memory extraction from conversations
- Integration with the vector store

We did not replace or redesign Mem0.

ChromaDB

ChromaDB stores:

- Actual memory text
- High-dimensional embeddings
- Vector-search information

Embeddings are not duplicated into SQLite.

SQLite research database

SQLite stores lightweight research metadata:

- Memory creation interaction
- Retrieval count
- Retrieval timestamps
- Retrieval ranks
- Similarity observations
- Aggregated usage statistics
- Chat sessions and messages

UAMR

UAMR calculates the estimated future usefulness of each memory and produces a retention recommendation.

5. How one chat request works

When the user submits a message:

1. FastAPI receives the request.

```
2. The Agent calls MemoryEngine.search_memories().
```

3. Mem0 converts the query into an embedding.
4. ChromaDB searches for semantically similar memories.
5. Mem0 returns ranked memories.
6. UsageTracker records the retrieval results in SQLite.
7. The retrieved memory text is added to the Agent's prompt.
8. Qwen3 generates the answer.
9. The response is streamed to React.
10. The interaction is passed to Mem0 for memory extraction.

11. The web response displays the exact memories used.

12. UAMR evaluates their current retention scores.

The Agent does not know how the research telemetry is stored. It only communicates with the MemoryEngine abstraction.

6. Research instrumentation

Why instrumentation was required

Mem0 can retrieve memories, but our research algorithm needs to know how each memory has historically behaved.

For example:

- Was the memory retrieved frequently?
- Was it retrieved recently?
- Was it usually ranked first or fifth?
- Did it have high semantic similarity?
- Is it almost identical to another stronger memory?

Therefore, every Mem0 search is observed by the instrumentation layer.

Retrieval event

For every retrieved memory, the system records:

```
event_id
memory_id
interaction_number
query
similarity_score
retrieval_rank
timestamp
```

For example:

Memory: m-123

Query: "What is my favourite food?"

Similarity: 0.91

Rank: 1

Interaction: 25

This becomes one retrieval event.

Aggregate usage data

The system also maintains per-memory aggregates:

```
retrieval_count
similarity_sum
```

```
rank_discounted_relevance_sum
average_similarity
average_retrieval_rank
last_retrieved_interaction
last_retrieved_at
```

The exact interception happens in:

- Mem0Engine.search_memories() - app/memory/engine.py
- UsageTracker.track_search() - app/research/tracker.py
- SQLiteUsageRepository.record_retrievals() - app/research/repository.py

7. The four UAMR signals

UAMR uses four signals:

```
F = Retrieval frequency
R = Recency
Q = Retrieval quality
D = Redundancy
```

The first three indicate usefulness. Redundancy is a penalty.

7.1 Retrieval frequency - F(m)

The formula is:

```
F(m) = (c_m + α) / (n_m + α + β)
```

Where:

- c_m is the number of times memory m was retrieved.
- n_m is the number of interactions since the memory was created.
- α and β are smoothing parameters.
- Default values are α = 1 and β = 1.

Name of the mathematical technique

This is based on Bayesian smoothing, specifically the posterior-mean form associated with a Beta-Bernoulli model.

It can also be explained as Laplace-style smoothing when both parameters are one.

Why smoothing is needed

Without smoothing:

```
F = retrieval count / age
```

A newly created memory could produce unstable results.

For example, if a memory is created and retrieved immediately:

```
retrieval count = 1
age = 1
```

```
F = 1 / 1 = 1.0
```

That would make the system extremely confident after only one observation.

With smoothing:

```
F = (1 + 1) / (1 + 1 + 1)
F = 2 / 3
F = 0.6667
```

This produces a more conservative result.

Interpretation

- High F: Frequently retrieved relative to its age.
- Low F: Rarely retrieved despite existing for many interactions.

Function used

The calculation is performed inside:

```
RetentionScorer.score()
```

in scorer.py` - app/retention/scorer.py.

7.2 Recency - R(m)

The formula is:

```
R(m) = exp(-Δ_m / τ)
```

Where:

- Δ_m is the number of interactions since the last retrieval.
- τ is the decay time constant.
- exp is the exponential function.

Name of the equation

This is called exponential decay.

It is commonly used in:

- Time-decay systems
- Cache policies
- Survival models
- Radioactive decay
- Recommender systems
- Forgetting models

Why we used exponential decay

A memory used recently is more likely to be useful again than a memory that has not been used for a long time.

Exponential decay provides a smooth reduction:

```
Δ = 0 -> R = 1.000
```

```
Δ = 1 -> slightly below 1
```

```
Δ = 10 -> much lower
```

Δ -> infinity -> R approaches 0

It never suddenly changes from useful to useless.

Meaning of tau

τ controls how quickly the memory becomes stale.

A small value:

```
τ = 3
```

causes rapid decay.

A larger value:

```
τ = 20
```

causes slower decay.

The default in our project is:

```
τ = 10 interactions
```

Never-retrieved memories

If a memory has never been retrieved, its creation interaction becomes the recency reference.

```
Δ = current interaction - creation interaction
```

A grace period protects newly created memories from being immediately pruned.

Function used

Implemented in:

```
RetentionScorer.score()
```

using:

math.exp(-delta / tau)

7.3 Retrieval quality - Q(m)

For every retrieval, we calculate:

```
q = s / log2(r + 1)
```

Where:

- s is semantic similarity.
- r is retrieval rank.
- log2 is the base-two logarithm.

The historical score is:

```
Q(m) = average(q across all retrievals)
```

If a memory has never been retrieved:

```
Q(m) = 0
```

Name of the technique

This is a logarithmic rank discount inspired by Discounted Cumulative Gain, or DCG.

It is not a complete DCG metric. We use the rank-discounting idea from information retrieval.

Why rank is important

Two memories might have the same similarity but different rankings.

Suppose similarity is 0.90.

At rank 1:

```
q = 0.90 / log2(1 + 1)
q = 0.90 / 1
q = 0.90
```

At rank 5:

```
q = 0.90 / log2(5 + 1)
q = 0.90 / 2.585
```

q approximately 0.348

A rank-one retrieval contributes more because it was more important to the search result.

Why similarity alone is insufficient

A memory could have moderate similarity and always appear near the top. Another could have similar scores but consistently appear last.

Rank helps distinguish their practical retrieval usefulness.

Functions used

The contribution is calculated when events are stored in:

```
SQLiteUsageRepository._record_observation()
```

The average is calculated in:

```
RetentionScorer.score()
```

using:

```
rank_discounted_relevance_sum / retrieval_count
```

7.4 Redundancy - D(m)

Before applying redundancy, the system calculates a preliminary value:

```
V(m) = wF*F(m) + wR*R(m) + wQ*Q(m)
```

Then it compares the memory only with memories having a higher preliminary value.

Similarity between embeddings is calculated using cosine similarity:

```
cosine(a,b) = (a * b) / (||a|| ||b||)
```

Name of the similarity equation

This is cosine similarity.

It measures the angle between two embedding vectors.

Typical interpretation:

```
1.0 = extremely similar direction
```

```
0.0 = unrelated direction
```

```
-1.0 = opposite direction
```

For semantic embeddings, higher values usually indicate more similar meanings.

Redundancy threshold

The default threshold is:

```
θd = 0.85
```

If similarity is below the threshold:

```
D(m) = 0
```

If similarity is above the threshold:

```
D(m) = (similarity - θd) / (1 - θd)
```

Name of this transformation

This is a thresholded linear normalization.

It can also be described as a hinge-like penalty:

- No penalty below the threshold.
- A linearly increasing penalty above the threshold.
- A similarity of one produces a redundancy score of one.

Example

Suppose:

```
similarity = 0.94
```

```
θd = 0.85
```

Then:

```
D = (0.94 - 0.85) / (1 - 0.85)
```

```
D = 0.09 / 0.15
```

```
D = 0.60
```

Why compare only against higher-value memories?

Imagine two almost identical memories:

```
Memory A: V = 0.80
```

```
Memory B: V = 0.45
```

```
Similarity = 0.96
```

If both were penalized equally, both could receive low scores and be pruned.

Instead:

- Memory A is the stronger representative.
- Memory B is compared against A.
- B receives the redundancy penalty.
- A is not penalized by the weaker B.

This makes redundancy handling asymmetric and preserves the best representative.

Function used

Implemented in:

```
RetentionScorer._maximum_redundancy()
```

Cosine similarity is calculated in:

```
RetentionScorer.cosine_similarity()
```

8. Final retention equation

The final equation is:

```
S(m) = wF*F(m) + wR*R(m) + wQ*Q(m) - λ*D(m)
```

Where:

- S(m) is the final retention score.
- wF is the frequency weight.
- wR is the recency weight.
- wQ is the retrieval-quality weight.
- λ controls the redundancy penalty.

Default values:

```
wF = 1/3
wR = 1/3
wQ = 1/3
λ = 0.25
```

Name of the equation

This is a weighted additive utility function with a subtractive penalty.

It combines multiple indicators into one decision score.

This kind of construction is used in:

- Multi-criteria decision making
- Ranking systems
- Search scoring
- Recommendation systems

- Resource-allocation policies

Why three positive terms and one negative term?

Frequency, recency, and quality provide evidence that a memory is useful:

+ wF*F

+ wR*R

+ wQ*Q

Redundancy indicates that the information may already be represented by another stronger memory:

- λ*D

Real calculation example

Suppose a memory has:

```
F = 0.5238
R = 0.5488
Q = 0.7527
D = 0.0000
```

Using the defaults:

```
S = 0.3333(0.5238)
```

+ 0.3333(0.5488)

+ 0.3333(0.7527)

- 0.2500(0.0000)

Therefore:

S approximately 0.1746 + 0.1829 + 0.2509 - 0

S approximately 0.6084

The Policy Engine generates this equation using the memory's real backend values.

9. How we developed our own equation

It is important to explain this accurately.

We did not invent Bayesian smoothing, exponential decay, logarithmic discounting, cosine similarity, or PCA. These are established mathematical techniques.

Our contribution is:

Selecting these techniques, adapting them to memory-retention telemetry, combining them into one UAMR scoring policy, and designing the asymmetric redundancy rule and budget-selection process.

The equation-design process

Step 1: Define the objective

We wanted to estimate:

How useful is this memory likely to be in future interactions?

There is no direct future-usefulness value available, so we needed observable proxy signals.

Step 2: Identify observable evidence

We selected:

- How often the memory was retrieved
- How recently it was retrieved
- How useful its historical retrieval positions were
- Whether another stronger memory already represented the same information

These became F, R, Q, and D.

Step 3: Keep signals on comparable scales

The signals are designed to be approximately between zero and one.

That prevents one component from dominating merely because it uses larger numerical units.

Step 4: Combine positive evidence

We used a weighted sum:

wF*F + wR*R + wQ*Q

A weighted sum is transparent and easy to explain, test, and configure.

Step 5: Add a duplication penalty

We subtracted:

λ*D

This allows useful but repetitive memories to lose value relative to stronger representatives.

Step 6: Separate scoring from the budget decision

The equation only calculates estimated value.

The BudgetManager independently decides how many memories can remain active.

This separation is important:

Scoring: How valuable is each memory?

Budgeting: How many can we retain?

Step 7: Add cold-start protection

New memories have not had enough time to accumulate usage.

Therefore, a grace period prevents the system from judging them too early.

Step 8: Make everything configurable

The weights are experimental, not claimed to be universally optimal.

Future experiments can compare different values of:

α, β, τ, G, wF, wR, wQ, λ, θd

10. Cold-start grace period

A new memory may initially have:

```
retrieval_count = 0
```

```
Q = 0
```

That does not necessarily mean it is useless. It may simply not have had a chance to be retrieved.

Therefore:

If memory age < G, it cannot be pruned.

Where G is the grace period.

Default:

```
G = 3 interactions
```

This technique is called cold-start protection.

Without it, newly created memories could immediately lose to older memories with established usage histories.

11. Memory-budget algorithm

After scores are calculated:

1. Sort memories by retention score in descending order.

2. Calculate the target:

```
K = ceil(N x B)
```

3. Include all grace-period-protected memories.

4. Fill remaining positions with the highest-scoring memories.

5. Label selected memories ACTIVE.

6. Label remaining memories PRUNED.

Function used

Implemented in:

```
BudgetManager.apply()
```

in budget.py` - app/retention/budget.py.

Example

For:

```
N = 20
```

```
B = 0.50
```

The target is:

```
K = ceil(20 x 0.50)
```

```
K = 10
```

Therefore:

```
ACTIVE = 10
```

```
PRUNED = 10
```

If more than 10 memories are protected by the grace period, the active count may exceed 10. This is intentional.

12. ACTIVE and PRUNED do not mean deletion

The policy is currently a simulation.

```
ACTIVE = would remain available under this budget
```

```
PRUNED = would be removed from the active set
```

No memory is physically deleted from Mem0 or ChromaDB.

This is important because:

- We can compare policies safely.
- We do not lose research data.
- We can repeat experiments with different budgets.
- Changing from 25% to 100% restores all simulated memories.

13. Deterministic explanations

Each decision receives a machine-generated but deterministic explanation.

Examples:

ACTIVE - high retrieval quality and recent usage

ACTIVE - protected by cold-start grace period

PRUNED - low estimated future usefulness under the budget

PRUNED - low relative value and high redundancy with memory memory-id

“Deterministic” means the explanation is assembled from rules. It is not generated by the LLM.

This makes the research output:

- Repeatable
- Auditable
- Easy to evaluate
- Free from LLM hallucination

The function is:

```
BudgetManager._reason()
```

14. PCA vector memory map

ChromaDB embeddings may contain hundreds of dimensions.

Humans cannot directly visualize a vector such as:

```
[0.021, -0.114, 0.074, ...]
```

Therefore, the project uses Principal Component Analysis, or PCA.

Name of the technique

PCA - Principal Component Analysis

PCA is a statistical dimensionality-reduction method.

It identifies directions that explain the largest amount of variation in the embedding data.

We reduce:

High-dimensional embeddings -> two coordinates

Each memory becomes:

```
(memory_id, x, y)
```

Main PCA operations

Mean centering

For every dimension, subtract the mean:

```
Xcentered = X - mean(X)
```

Singular Value Decomposition

The implementation uses:

```
numpy.linalg.svd()
```

SVD decomposes the centered matrix and identifies its main directions.

Projection

The centered embeddings are projected onto the first two principal components:

```
Z = Xcentered x principal components
```

The resulting x and y values are sent to React.

Why PCA?

We chose PCA because it is:

- Lightweight
- Deterministic
- Fast for the current dataset
- Suitable for an 8 GB laptop
- Easy to explain academically
- Available through NumPy

We did not use t-SNE or UMAP because they add dependencies, cost, and parameter sensitivity that are unnecessary for the initial visualization.

Important limitation

Two memories appearing close in the map often indicates similarity, but 2D distance does not perfectly preserve the original high-dimensional relationship.

The actual redundancy algorithm uses cosine similarity on the original embeddings-not PCA coordinates.

Therefore:

PCA affects only visualization. It does not affect UAMR scores or decisions.

The implementation is in:

```
PCAProjectionCache.project()
```

```
PCAProjectionCache._compute()
```

in research_visualization.py` - app/services/research_visualization.py.

15. Why PCA is cached

Changing the budget affects only policy states:

```
ACTIVE / PRUNED
```

It does not change the memory embeddings.

Therefore, recalculating PCA whenever the slider changes would be wasteful.

The server generates a fingerprint from the memory IDs and embeddings. If the fingerprint has already been projected, it reuses the cached coordinates.

This improves performance while ensuring that updated embeddings create a new projection.

Only the following reach the browser:

memory ID

memory text

x coordinate

y coordinate

retention score

```
state
```

Raw embeddings remain on the server.

16. Memory Lab explanation

The Memory Lab is an evidence-exploration interface, not a CRUD dashboard.

It shows:

- Total memories
- Active memories
- Pruned memories

- Current budget
- Average retention score
- Average retrieval count
- Redundancy rate

The redundancy rate is:

Number of memories with D > 0

Total number of memories

Because D > 0 only when similarity exceeds the configured redundancy threshold, this represents the fraction receiving a redundancy penalty.

Memory table

Every row shows:

Memory text

F
R
Q
D
S

Retrieval count

Policy state

The table can be sorted by any score.

Filters include:

- All
- Active
- Pruned
- Highly redundant
- Never retrieved

Detail drawer

Clicking a row or PCA point shows:

- Complete text
- Memory ID
- Creation timestamp
- Last-retrieval timestamp
- Retrieval count
- Average similarity

- Average rank
- F/R/Q/D values
- Exact retention equation
- Current state
- Deterministic explanation
- Complete retrieval history
- Five nearest semantic memories

Frontend implementation:

- MemoryLabPage.tsx` - frontend/src/pages/MemoryLabPage.tsx
- MemoryDetailDrawer.tsx` - frontend/src/components/MemoryDetailDrawer.tsx
- MemoryMap.tsx` - frontend/src/components/MemoryMap.tsx

17. Policy Engine explanation

The Policy Engine explains how UAMR reaches a decision.

It displays:

F(m) - Retrieval Frequency

R(m) - Recency

Q(m) - Retrieval Quality

D(m) - Redundancy

Then the final formula:

```
S(m) = wF*F + wR*R + wQ*Q - λ*D
```

For a selected memory, horizontal bars show the magnitude of each signal.

The displayed equation is generated using values returned by FastAPI. No demonstration values are hardcoded.

Budget simulator

The slider supports:

100%, 75%, 50%, 25%

Changing it sends a new request to the backend and recalculates:

- Number active
- Number that would be pruned
- Lowest active score
- Table decisions
- PCA-map states
- Selected-memory decision

The PCA coordinates are reused because changing the budget does not change the embeddings.

Frontend implementation:

- PolicyEnginePage.tsx` - frontend/src/pages/PolicyEnginePage.tsx
- ScoreExplanation.tsx` - frontend/src/components/ScoreExplanation.tsx
- MemoryScoreTable.tsx` - frontend/src/components/MemoryScoreTable.tsx

18. Important backend classes and functions

Component	Function	Purpose
Mem0Engine	search_memories()	Performs the real Mem0 search
UsageTracker	track_search()	Converts Mem0 results into retrieval observations
SQLiteUsageRepository	record_retrievals()	Stores events and updates aggregates
Mem0RetentionDataSource	load()	Joins Mem0 memories, Chroma embeddings, and SQLite usage
RetentionScorer	score()	Calculates F, R, Q, D, and S
RetentionScorer	_maximum_redundancy()	Finds the strongest redundant representative
RetentionScorer	cosine_similarity()	Calculates original-space embedding similarity
UAMRPolicy	evaluate()	Runs scoring and budget selection
BudgetManager	apply()	Assigns ACTIVE and PRUNED
BudgetManager	_reason()	Produces deterministic explanations
PolicyService	evaluate()	Provides the policy through a service layer
ResearchVisualizationService	overview()	Produces summary, table, and map data
ResearchVisualizationService	detail()	Produces complete evidence for one memory
PCAProjectionCache	project()	Returns cached 2D coordinates
TracedMemoryEngine	search_memories()	Captures memories used for a web response
FastAPI route	stream_chat()	Streams response tokens and memory traces

19. What is original in the project?

A good academic answer is:

> **The individual mathematical functions are established methods. Our original work is their integration into a transparent memory-retention policy for Mem0-based agents.**

The project-specific contributions are:

- Recording usage evidence without replacing Mem0
- Separating embeddings from research metadata
- Combining frequency, recency, retrieval quality, and redundancy
- Using rank-discounted semantic quality

- Comparing redundancy only against higher-value memories
- Cold-start protection
- Non-destructive budget simulation
- Deterministic explanations
- Per-response memory traces
- Interactive evidence and PCA visualization
- Configurable experimental parameters

Do not tell your guide that you invented Bayesian smoothing, cosine similarity, exponential decay, or PCA.

Instead say:

> **We designed the UAMR composite equation by adapting established mathematical techniques to the long-term memory-retention problem.**

20. Current limitations

Mentioning limitations makes the project sound more scientific.

- The initial weights are manually selected experimental defaults.
- The weights have not yet been optimized through benchmark experiments.
- Retrieval frequency assumes past retrieval is evidence of future usefulness.
- Semantic similarity does not always mean factual duplication.
- PCA loses some high-dimensional information.
- Current decisions are simulated rather than physically enforced.
- The dataset is currently relatively small.
- Retrieval behavior depends partly on Mem0 and the embedding model.
- No reinforcement learning or learned retention model is used.

21. Future experimental work

Possible future work includes:

- Compare UAMR against recency-only retention.
- Compare it against frequency-only retention.
- Compare it against random pruning.
- Test multiple memory budgets.
- Measure answer accuracy after simulated pruning.
- Measure retrieval latency.
- Measure redundant-memory reduction.
- Tune weights using validation data.
- Evaluate sensitivity to τ, λ, and θd.
- Introduce safe physical archiving only after simulation results are validated.

Useful evaluation metrics could include:

Answer accuracy

Memory recall

Retrieval precision

Redundancy reduction

Average prompt size

Retrieval latency

Important-memory preservation rate

22. Suggested final presentation statement

UAMR is a transparent, configurable, and non-destructive retention policy for long-term AI-agent memory. It does not replace Mem0 or use another machine-learning model to make retention decisions. Instead, it observes actual memory usage and combines four interpretable signals: frequency, recency, retrieval quality, and redundancy. The resulting score allows the system to rank memories under different capacity budgets while protecting new memories and preserving stronger representatives of duplicate information. The research interface makes every decision inspectable through equations, retrieval histories, nearest memories, and a PCA embedding map.